

Automatización de importación FBX en Bannerlord

Investigación técnica | 23 de septiembre de 2026 | Calradia Forge

1. Decisión recomendada

Construir un orquestador externo de Windows en C#, con un adaptador UI Automation para la versión instalada de Resource Browser. Su viabilidad depende de inspeccionar los controles reales y de verificar la salida después del envío. La investigación no establece todavía una importación desatendida de extremo a extremo.

La búsqueda en la documentación de recursos de TaleWorlds y la consulta puntual de Utilities en la API 1.2.12 no identificaron una API pública documentada de importación FBX por lotes. Es un resultado limitado de búsqueda, no una prueba de que no existan herramientas internas. No basar el desarrollo en comandos, flags o sidecars supuestos. [1, 2, 10]

Hechos que sí sustentan el diseño:

- TaleWorlds documenta abrir Resource Browser mediante `resource.show_resource_browser` desde la consola del Modding Kit. La apertura es distinta de la importación. [1]
- Los nombres internos del FBX agrupan mallas y LOD. Se admiten `.lodN` y, como alternativa, `_lodN`; `bo_` identifica formas físicas. Un FBX puede producir varios recursos. [2]
- Importar geometría no crea sus materiales: deben existir con los nombres referenciados. Las texturas se importan por separado y se asignan a los materiales. [2, 4]
- `Assets` contiene TPAC editables, `AssetSources` las fuentes y `AssetPackages` los paquetes de cliente. Mantener el árbol de autoría y preparar la distribución en una copia aparte. [3]

Separar tres perfiles: modelos estáticos, mallas con esqueleto/animaciones y texturas. `SpriteSheetGenerator` corresponde a atlas de interfaz y requiere un flujo propio; no es un compilador FBX. [1]

La guía práctica de su autor en BannerlordModding.LT muestra esta secuencia: menú contextual, selección de FBX, ajustes e Import, revisión del resultado. Es evidencia comunitaria directa, no un contrato estable para todas las versiones. [5]

Flujo propuesto: inventario > destino confirmado > envío > ajustes e importación > verificación del recurso. Una incertidumbre detiene el lote y conserva el diagnóstico.

2. Correcciones al texto adjunto

Lectura de procesos. `StandardOutput.ReadToEnd()` es síncrono, aunque el comentario del ejemplo diga lo contrario. Si el hijo mantiene stdout abierto mientras espera entrada, el padre no llegará al `StandardInput.WriteLine()`: ambos pueden quedar esperando. Es una deducción del orden del código y de la semántica de .NET. [8]

Para un ejecutable cuyo protocolo se haya comprobado: consumir stdout y stderr concurrentemente, responder a la petición real de entrada y esperar la salida con plazo y cancelación. Cambiar a lectura asíncrona no demuestra que `SpriteSheetGenerator` acepte stdin. No se comprobó ese protocolo en su ejecutable instalado.

Bloqueos de PNG. La guía oficial no documenta la afirmación de que ENTER libere un bloqueo especial. Finalizar un proceso normalmente libera sus handles; una terminación abrupta puede dejar escrituras incompletas. No confundir archivos incompletos con un bloqueo persistente ni automatizar la pausa sin reproducirla.

Carpetas. El texto dice a la vez que deben conservarse y que su coexistencia provoca una caída. `TaleWorlds` explica sus funciones, pero la fuente consultada no prueba esa causa de fallo. El nombre oficial es `AssetPackages`, en plural. [3]

Modales. Cerrar toda ventana recibida por `WindowOpenedEvent` puede descartar opciones de importación, confirmaciones de sustitución o errores relevantes. Tampoco garantiza ver todos los diálogos si son ventanas superiores o de otro proceso. Registrar identidad, texto y propietario; detener ante desconocidos. Recuperar automáticamente solo casos ensayados y clasificados.

Esqueletos. `_notused` indica que no se importe ese esqueleto al importar recursos relacionados. `human_skeleton_notused` no es una regla universal para todo FBX ni una orden documentada de transferencia de pesos. La jerarquía y numeración compatible de huesos importan cuando hay archivos separados. [2]

Leaf Bones. La guía de exportación de su autor recomienda desactivarlos en su flujo de Blender. No demuestra la explicación interna del skinning que afirma el adjunto ni justifica modificar automáticamente todos los FBX. [6]

Foco y espera. `SetForegroundWindow`, `SendKeys` y pausas fijas no acreditan que la consola recibió el comando. Elegir ventana y controles observados, verificar foco cuando haya teclado y esperar condiciones comprobables con plazo.

3. Arquitectura para programarlo

Esta sección es una propuesta de ingeniería, todavía no una capacidad comprobada del editor.

Planificador. Inventariar rutas canónicas, tamaño y SHA-256; limitar archivos, bytes y profundidad; no seguir enlaces de directorio. Registrar tipo de recurso, nombres internos esperados, dependencias de materiales, módulo y carpeta destino. Conservar fuentes intactas. Comparar nombres internos contra el destino: nombres de archivo distintos no impiden una colisión de recursos.

Adaptador Windows. Usar C# con FlaUI como opción práctica de acceso a UIA2/UIA3, o conservar `System.Windows.Automation` si los patrones necesarios funcionan. FlaUI es una envoltura de UIA, no añade accesibilidad al editor. Los controles personalizados necesitan un proveedor adecuado; medir primero qué expone Resource Browser. [7, 9]

Guardar un perfil de selectores por versión e idioma: proceso/ventana, ruta del módulo, menú de importación, selector de archivo, ajustes, acción Import y condición de finalización. Combinar tipo de control, identidad y ámbito; exigir coincidencia única. No deducir IDs desde capturas ni reutilizar índices efímeros.

Si los controles no son accesibles, conservar una importación asistida. Un adaptador visual es una alternativa con mayor dependencia de resolución, escala y diseño; no debe presentarse como equivalente en robustez. Playwright y Selenium automatizan navegadores web y no sustituyen un adaptador para este editor nativo.

Máquina de estados propuesta:

- **PLANNED:** fuentes y destino esperados registrados.
- **TARGET_READY:** proceso y carpeta observados de forma inequívoca.
- **SUBMITTED:** ruta enviada al selector; todavía no implica importar.
- **IMPORTING:** ajustes comprobados y acción Import enviada.
- **OUTPUT_OBSERVED:** el editor terminó y hay evidencia atribuible de salida.
- **VERIFIED:** nombres/tipos y dependencias esperados revisados; prueba visual registrada por separado.
- **STOPPED:** fallo conocido; **UNKNOWN:** no se sabe si la acción produjo efectos.

Control del trabajo. Una sola importación activa por editor. Usar eventos como señales y volver a consultar el estado antes de avanzar. Un plazo en un bucle no interrumpe un `Invoke()` bloqueado: aislar el adaptador en un proceso trabajador permite que el supervisor detecte su bloqueo. Detener el trabajador deja la operación del editor en estado desconocido; no reenviarla automáticamente.

Registro y reanudación. Escribir JSONL por transición con hash, versión, destino, hora UTC, ajustes, texto de errores y evidencia de salida. Reanudar solo tras reconciliar archivos y recursos existentes. Una política de sustitución debe ser explícita y quedar registrada.

4. Lo que ya existe en tu proyecto

Se leyeron los archivos actuales del espacio de trabajo, sin modificar los importadores.

Helper mantenido: `BannerlordImportAutomation.csproj`, documentado en `docs/BANNERLORD_IMPORT_AUTOMATION.es.md`. Usa FlaUI 5.0.0 y distingue inspección, simulación y envío experimental. El prototipo `BannerlordEditorImport.csproj` queda fuera de su validación normal según esa documentación.

Experimento FBX: `BannerlordFbxImporter/`. Su configuración deja vacías las identidades de los controles. Ofrece `--dry-run`, `--inspect` y `--submit`; contiene inventario acotado, resolución de destino y detención ante estados inciertos.

Hallazgos del código `WindowsEditorAutomation.cs` y `SubmissionCoordinator.cs`:

- Selecciona el nodo `Assets`, invoca el control configurado, rellena el selector y confirma el archivo.
- Su final satisfactorio es `SUBMITTED`; no valida TPAC, geometría ni carga en el juego.
- Falta modelar explícitamente la pantalla de ajustes y la acción final de importación descritas por la guía práctica. Un diálogo adicional puede detener el helper actual. [5]
- Se conecta a `Process.MainWindowHandle` y rechaza otras ventanas visibles del proceso. Hay que comprobar si Resource Browser es una ventana secundaria legítima y ajustar el modelo de propiedad si corresponde.
- La ruta directa `módulo > Assets` es una expectativa del helper que debe contrastarse con el árbol real; no es un contrato UIA oficial.
- Los límites de tiempo alrededor de la espera no acotan las llamadas síncronas a `Invoke()`.

Verificación de esta investigación: `Test-BannerlordFbxImporter.bat` compiló sin errores ni advertencias y pasó 59 aserciones de inventario, preflight, CLI, destino y detención. Son pruebas aisladas, no una prueba del editor.

Computer Use enumeró las ventanas abiertas: no apareció Resource Browser ni una ventana del editor de Bannerlord. No se inició el editor ni se importó ningún recurso. Por ello no se obtuvieron selectores ni patrones UIA reales.

Se comprobó la presencia del generador, launcher y `TaleWorlds.Engine.dll` en `Win64_Shipping_wEditor`. Sus metadatos `FileVersion` devolvieron 1.0.0.0, insuficiente para identificar la versión comercial instalada. La referencia API 1.2.12 consultada tampoco verifica compatibilidad con esta instalación.

Recomendación: ampliar un único helper existente con perfiles, estados de importación y verificación. Evitar crear un tercer importador que repita solo el envío del archivo.

5. Experimento mínimo y criterios de aceptación

Antes de implementar el envío completo, documentar una importación manual reproducible en un módulo de prueba. Registrar versión del juego/Modding Kit, idioma, ruta de instalación y FBX de entrada. Preparar un modelo estático pequeño, sin colisiones de nombres, y sus materiales.

1. Abrir Resource Browser y observar tanto la ventana principal como las ventanas superiores de su proceso. Registrar selectores y patrones de los controles que intervienen en cada paso.
2. Recorrer manualmente menú de importación, selector, ajustes, avisos y resultado. Identificar el momento que realmente autoriza la compilación. Repetir para detectar estados variables.
3. Implementar el recorrido de un archivo con registro de estados. Comprobar la carpeta seleccionada antes de enviar y después de cualquier cambio de UI.
4. Tomar inventario de salida antes/después. Un hash o tamaño cambiado es evidencia de escritura, no prueba de un recurso correcto. Correlacionar nombres y tipos esperados con el catálogo del editor o un lector compatible.
5. Revisar el modelo en Model Viewer: escala, orientación, materiales, LOD y, cuando corresponda, rig/animación. Reiniciar y volver a abrir el recurso para comprobar persistencia.
6. Ampliar a un lote pequeño solo después de validar el primer archivo. Medir duración y errores reales; no prometer ganancias de rendimiento sin esas mediciones.

Casos de aceptación necesarios:

- Material inexistente: no se reporta éxito completo ni se oculta el aviso.
- Nombre interno duplicado: se aplica la política prevista antes de una sustitución.
- FBX inválido o binario no analizado: se distingue evidencia no disponible de archivo válido.
- Ventana inesperada, pérdida de foco o proceso bloqueado: se detiene sin enviar el siguiente archivo.
- Reinicio del orquestador tras enviar: se reconcilia el resultado antes de reintentar.
- Dos FBX con varios recursos cada uno: se verifica el conjunto esperado, sin asumir un archivo de salida por entrada.

TpacTool puede ayudar a inspeccionar/exportar cuando sea compatible. Su proyecto declara la importación y edición sin implementar; no es un sustituto del compilador oficial ni su lectura demuestra renderizado correcto. [11]

Resultado de la investigación: hay una base de código aprovechable. El siguiente hito demostrable es calibrar y validar una importación real de un FBX, incluyendo ajustes y resultado. La ejecución por lotes desatendida queda condicionada a superar ese hito.

6. Fuentes y alcance

Consulta realizada el 23-09-2026. Las fuentes comunitarias siguientes son guías prácticas de sus autores; se identifican como tales. No se atribuyen sus observaciones a TaleWorlds.

- [1] TaleWorlds. [Generating and Loading UI Sprite Sheets](#). Flujo de sprites y comando para abrir Resource Browser.
- [2] TaleWorlds. [Naming Conventions](#). LOD, materiales, cuerpos y sufijo de esqueleto.
- [3] TaleWorlds. [Adding & Overriding Assets](#). Carpetas, paquetes y sustitución por nombres.
- [4] TaleWorlds. [Textures](#). Importación de texturas y asignación a materiales.
- [5] BannerlordModding.LT. [FBX import into Editor](#). Recorrido práctico de importación y ejemplos de problemas.
- [6] BannerlordModding.LT. [Export to FBX](#). Ajustes de exportación de Blender en el flujo descrito por el autor.
- [7] FlaUI. [Repositorio y documentación del proyecto](#). Alcance y diferencias de UIA2/UIA3.
- [8] Microsoft. [Process.StandardOutput](#). Lectura síncrona y dependencias entre streams/proceso.
- [9] Microsoft. [UI Automation Providers Overview](#). Exposición de controles estándar y personalizados.
- [10] TaleWorlds. [Utilities, API 1.2.12](#). Consulta puntual; no es inventario exhaustivo de APIs privadas o actuales.
- [11] TpacTool. [Repositorio y matriz de capacidades](#). Visor/exportador y limitaciones de importación.

Fuentes locales: texto adjunto del usuario; docs/BANNERLORD_IMPORT_AUTOMATION.es.md; BannerlordFbxImporter/README.es.md, appsettings.json, Program.cs, WindowsEditorAutomation.cs, SubmissionCoordinator.cs y DialogCloseGate.cs. La prueba ejecutada fue Test-BannerlordFbxImporter.bat. El inventario de ventanas se obtuvo con Computer Use en esta sesión.

Alcance: investigación documental, lectura del código y prueba aislada existente. No se probaron la importación real, el protocolo de consola del generador, la compilación TPAC ni la representación visual en Bannerlord.